Plan Maestro Definitivo y Ampliado: Plataforma Optimizadora de Presupuesto ("Yo Merco" - Colombia)

1. Resumen Ejecutivo y Visión del Producto

"Yo Merco" es un optimizador inteligente de presupuesto y comparador multicanal diseñado para el mercado colombiano (con enfoque estratégico en ciudades intermedias como Manizales y expansión nacional). La plataforma permite a los usuarios maximizar su poder adquisitivo mediante dos modalidades principales:

1. **Modo Lista Exacta:** Ingresar una lista detallada de productos y cantidades para descubrir de inmediato en cuál de las grandes superficies o tiendas de descuento sale más económico el mercado completo.
2. **Modo Presupuesto Inteligente:** Indicar un monto máximo disponible de dinero para que la app calcule el alcance exacto de la canasta en cada establecimiento o sugiera alternativas de distribución.

---

2. Supermercados y Canales Incorporados en el Ecosistema

Para garantizar una cobertura de mercado integral que abarque desde el ahorro de descuento duro hasta las opciones prémium y tradicionales de la ciudad, el comparador integra las siguientes 6 grandes marcas:

1. **Tiendas D1** (Descuento duro - Máxima presencia y economía en abarrotes).
2. **Tiendas Ara** (Descuento duro - Competidor directo con alta variedad en frescos y despensa).
3. **Mercaldas** (Supermercado regional líder en Caldas - Tradición, surtido local fuerte).
4. **Supermercado del Centro** (Comercio tradicional urbano - Referente en economía de plaza y víveres).
5. **Éxito** (Hipermercado tradicional / Grupo Éxito - Gran variedad, marcas propias y cobertura masiva).
6. **Carulla** (Supermercado gama alta / Grupo Éxito - Ideal para comparar segmentos prémium y atención especializada).

---

3. Alcance del MVP y Funcionalidades Avanzadas (Roadmap)

Fase 1: Núcleo del MVP (Funcionalidades Esenciales)

- **Gestión de Canasta:** Interfaz web ágil tipo carrito/lista interactiva con autocompletable de productos normalizados.
- **Motor de Cálculo Comparativo:** Algoritmo instantáneo basado en Supabase que procesa costos totales por supermercado.
- **Vista de Podio de Ahorros:** Visualización clara del 1er lugar (Ganador del ahorro), 2do y 3er lugar, mostrando la diferencia exacta en pesos y porcentaje respecto al mercado más costoso.
- **Catálogo Maestro Inicial:** Surtido inicial de ~50 productos esenciales de alta rotación (canasta básica colombiana).

Fase 2: Automatización y Datos (El motor detrás de bambalinas)

- **Sistema de Scraping Híbrido (Python + Playwright):** Extracción programada de precios por tienda.
- **Orquestación con n8n:** Automatización de tareas periódicas para actualización de precios.
- **Panel de Control Administrativo (CMS Interno):** Interfaz para supervisar precios desactualizados o subir cargas rápidas vía CSV si una página web cambia su estructura.

Fase 3: Funcionalidades Pro (Escalabilidad y Comunidad)

- **Modo "Supermercado Híbrido":** Sugerencia algorítmica avanzada para comprar el aceite y el arroz en D1, los lácteos en Ara y el aseo en Mercaldas (si la cercanía y el ahorro neto lo justifican tras restar costos de transporte).
- **Alertas de Ahorro / Historial de Precios:** Gráficas de tendencia para ver si un producto subió o bajó de precio respecto a la quincena pasada.
- **Reporte Colaborativo (Crowdsourcing):** Permisos para que la comunidad valide o reporte precios escaneando el código de barras o subiendo la foto del recibo/etiqueta física.

---

4. Arquitectura Tecnológica y Stack Integral

- **Frontend:** Next.js (React) con Tailwind CSS (Enfoque Mobile-First, ultra rápido, responsivo para uso rápido desde el celular en la entrada del supermercado).

- **Backend y Base de Datos: Supabase (PostgreSQL)** para gestión relacional de usuarios, listas, comercios y precios vigentes.
- **Motor de Extracción: Python 3.10+** con **Playwright** (para renderizado dinámico de tiendas online) y BeautifulSoup.
- **Orquestador: n8n** self-hosted para disparar webhooks y scripts de actualización periódica.

---

5. Diseño de Base de Datos Estructurada (Supabase / PostgreSQL)

1. Tabla: `supermarkets`

Almacena la información institucional de las 6 cadenas evaluadas.

- `id` (UUID, Primary Key)
- `name` (VARCHAR) - Ej: "Tiendas D1", "Tiendas Ara", "Mercaldas", "Supermercado del Centro", "Éxito", "Carulla"
- `slug` (VARCHAR) - Identificador único de URL (ej: `d1`, `ara`, `mercaldas`)
- `logo_url` (TEXT)
- `tier` (VARCHAR) - Ej: "hard_discount", "regional", "hypermarket", "premium"
- `created_at` (TIMESTAMP)

2. Tabla: `categories`

- `id` (UUID, Primary Key)
- `name` (VARCHAR) - Ej: "Despensa y Granos", "Lácteos y Huevos", "Frutas y Verduras", "Carnes y Pollo", "Aseo Hogar", "Cuidado Personal"

3. Tabla: `products`

- `id` (UUID, Primary Key)
- `name` (VARCHAR) - Ej: "Arroz Blanquita / Roa 1000g"
- `category_id` (UUID, Foreign Key -> `categories.id`)
- `unit_of_measure` (VARCHAR) - Ej: "1000g", "litro", "unidad", "paquete"
- `barcode` (VARCHAR, Nullable) - Código de barras EAN para futura lectura móvil
- `image_url` (TEXT)

4. Tabla: `supermarket_prices`

- `id` (UUID, Primary Key)
- `product_id` (UUID, Foreign Key -> `products.id`)

- `supermarket_id` (UUID, Foreign Key -> `supermarkets.id`)
- `price` (DECIMAL) - Precio en pesos colombianos (COP)
- `is_promo` (BOOLEAN) - Indicador si el precio aplica por oferta/descuento vigente
- `updated_at` (TIMESTAMP)

5. Tabla: `user_lists` (Gestión temporal o guardada del usuario)

- `id` (UUID, Primary Key)
- `user_id` (UUID, Nullable para invitados)
- `product_id` (UUID, Foreign Key -> `products.id`)
- `quantity` (DECIMAL)

---

6. Especificación de Experiencia de Usuario (UX/UI Flow)

1. **Landing Page / Selector de Ciudad:**
    - Bienvenida limpia con la propuesta de valor: *"No pagues de más en tu mercado. Comparamos D1, Ara, Mercaldas, Centro, Éxito y Carulla en segundos."*
    - Selección de ciudad (Por defecto: Manizales).
2. **Constructor de Canasta Inteligente:**
    - Barra de búsqueda rápida de productos con autocompletado y categorías visuales.
    - Contador flotante de la cantidad de artículos añadidos.
    - Opción rápida de alternar a *"Modo Presupuesto"*, donde el usuario ingresa su monto disponible (Ej. $200.000 COP) y la app ajusta o sugiere canastas óptimas.
3. **Pantalla de Resultados y Podio de Ahorro:**
    - **Tarjeta Campeona (1er Lugar):** Destacada visualmente con confeti o color verde esmeralda. Muestra el supermercado más económico, el valor total exacto, el ahorro frente al promedio y el dinero sobrante (si aplica presupuesto).
    - **Tarjetas de Comparativa (2° a 6° lugar):** Ordenadas de menor a mayor precio con barras de progreso relativas al costo.
    - **Desglose Detallado por Producto (Acordeón desplegable):** Una tabla transparente donde el usuario ve cuánto vale cada ítem en cada tienda, destacando en verde el precio más barato individual.

---

7. Motor de Cálculo Financiero (Consulta SQL de Optimización)

Para calcular en tiempo real el costo total de la canasta por cada uno de los 6 supermercados, se ejecuta la siguiente función optimizada en Supabase:

```
SELECT
    s.id as supermarket_id,
    s.name as supermarket_name,
    s.logo_url,
    SUM(sp.price * ul.quantity) as total_market_cost,
    COUNT(sp.product_id) as matched_products_count
FROM
    user_lists ul
JOIN
    supermarket_prices sp ON ul.product_id = sp.product_id
JOIN
    supermarkets s ON sp.supermarket_id = s.id
GROUP BY
    s.id, s.name, s.logo_url
ORDER BY
    total_market_cost ASC;
```

---

8. Guía Técnica de Web Scraping y Automatización (Python + Playwright)

Debido a la diversidad de plataformas (desde descuentos duros hasta hipermercados), el script en Python implementa una estrategia modular por cada supermercado utilizando **Playwright** para la navegación dinámica y **Supabase Python Client** para la persistencia automática de los precios.

```
import os

import time

import random

from playwright.sync_api import sync_playwright

from supabase import create_client, Client

SUPABASE_URL = os.environ.get("SUPABASE_URL", "TU_SUPABASE_URL")

SUPABASE_KEY = os.environ.get("SUPABASE_KEY", "TU_SUPABASE_SERVICE_KEY")

supabase: Client = create_client(SUPABASE_URL, SUPABASE_KEY)

# Mapeo de cadenas e hipermercados evaluados

SUPERMARKET_MAP = {

    "d1": {"id": "uuid-d1", "base_url": "https://tiendasd1.com/"},

    "ara": {"id": "uuid-ara", "base_url": "https://www.tiendasara.com/"},

    "mercaldas": {"id": "uuid-mercaldas", "base_url": "https://www.mercaldas.com/"},

    "centro": {"id": "uuid-centro", "base_url": "https://supermercadodelcentro.com/"},

    "exito": {"id": "uuid-exito", "base_url": "https://www.exito.com/"},

    "carulla": {"id": "uuid-carulla", "base_url": "https://www.carulla.com/"}

}

def get_stealth_context(browser):

    user_agents = [

        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36",

        "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/17.2.1 Safari/605.1.15"
```

```
    ]
    context = browser.new_context(
        user_agent=random.choice(user_agents),
        viewport={"width": 1366, "height": 768}
    )
    return context
def run_scraper():
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        context = get_stealth_context(browser)
        page = context.new_page()
        # Ejemplo de iteración de actualización de precios para productos claves
        print("Iniciando ciclo de actualización de precios multitienda...")

        # Lógica de navegación, extracción de precios mediante selectores robustos
        # y guardado automático con upsert en Supabase...
        browser.close()
if __name__ == "__main__":
    run_scraper()
```

---

9. Plan de Implementación Ágil por Fases (Listo para IA)

- **Fase 1 (Base de Datos & Esquema):** Configurar en Supabase las tablas `supermarkets`, `categories`, `products`, `supermarket_prices` y poblar las 6 marcas oficiales (`d1`, `ara`, `mercaldas`, `centro`, `exito`, `carulla`).
- **Fase 2 (Backend & Motor SQL):** Programar las vistas o consultas agregadas en PostgreSQL para devolver los totales por supermercado de forma instantánea.
- **Fase 3 (Frontend & UI Mobile First):** Desarrollar la interfaz con Next.js y Tailwind CSS, implementando el buscador interactivo de la canasta básica y el sistema de tarjetas de podio.
- **Fase 4 (Automatización & Ingesta):** Desplegar el script de Python en conjunto con flujos en **n8n** para asegurar que los precios se mantengan frescos quincena a quincena.
- **Fase 5 (Lanzamiento MVP):** Pruebas de campo en Manizales y despliegue en Vercel.